Coursework Diet 1
DevOps from Code to Application Deployment

1 Overview

In this coursework you will work in a team to improve, test and deploy a web application through a complete DevOps Continuous Integration/Continuous Deployment (CI/CD) pipeline using all of the tools and platforms you have learned in this module. These tools and platforms include AWS academy/platform, Git/GitHub, GitHub Actions, Docker/DockerHub, Unittest, Ansible and Kubernetes. The goal is to develop a system similar to the diagram below:

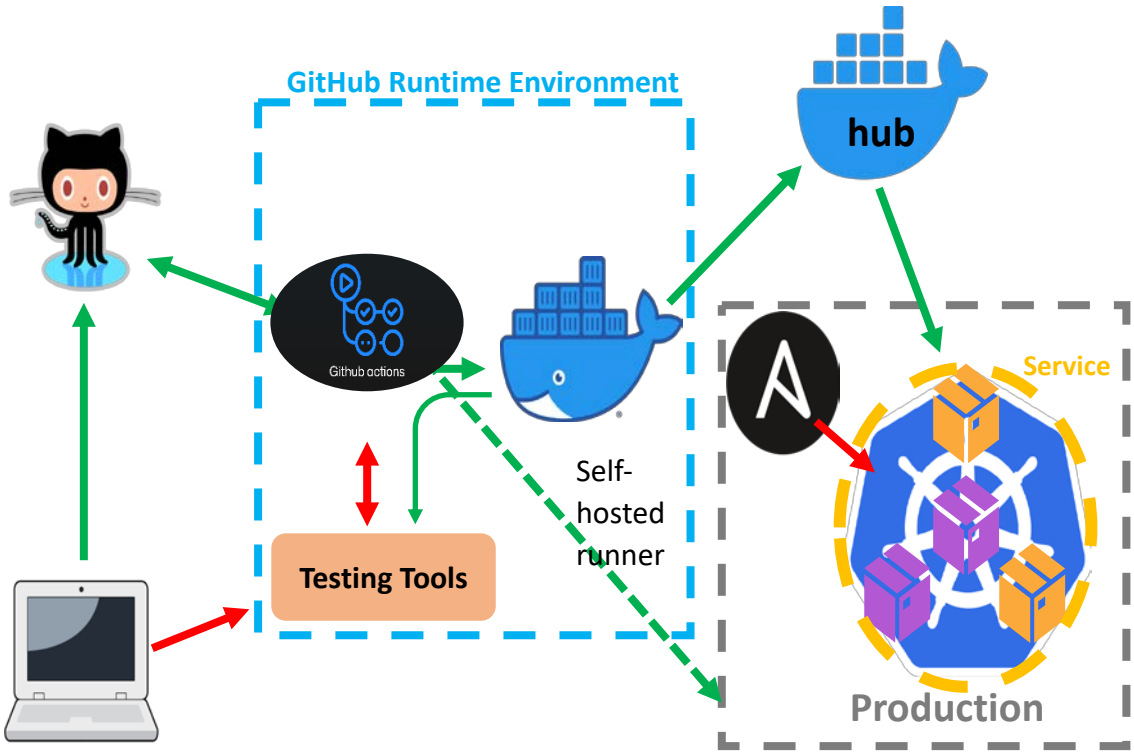


Figure 1 – DevOps CI/CD Pipeline Overview

The web application is coded using Flask framework in Python and displays an initial page that requires weather related measurements such as temperature, pressure, humidity etc., then uses these as inputs to a traditional machine learning model (Random Forest) that classifies the weather into 9 categories namely known as: clear, cloudy, drizzly, foggy, hazey, misty, rainy, smokey, thunderstorm. The classification result is then displayed on the web application. The project code integrates two components: web application code and the machine learning code. The code for the application uses Flask and Python3 and is **available on GCU Learn -> Assignments and Tests -> Coursework but is not**

complete and has some mistakes. You should complete the files as specified in the tasks in Practical Tasks section.

Submission:
There will be two submission portals on GCULearn to award the total marks for each task:

- Practical Evidence Report (**use the provided report template**) - Submitted via Turnitin Assignment
- Video demonstration - Submitted via GCU Learn
- Peer Evaluation form (optional, see document for guidance)

Both components of this coursework should be submitted to GCULearn no later than:

Thursday 4th December at 17:00

Work submitted must be yours alone and any suspected collusion or plagiarism will be addressed in line with the University Assessment Regulations.

2 Guidance

For the practical element of this coursework you are asked to implement a DevOps Continuous Delivery pipeline as outlined in **Figure 1** .

You should complete the Preliminary Steps **(see Preliminary Steps)** before proceeding.

The table below **(Practical Tasks)** provides an outline of the steps required to complete the practical element of this coursework along with the marks available for each component.

Marks indicated are for a complete working solution to the task as well as explanation in the video demonstration, and proportionate marks will be awarded for partial solutions.

Upon completion of these tasks, you should record a demonstration video of this pipeline following the guidance **(see Practical Evidence, Video Demonstration)** in this specification.

Please read section Practical Evidence carefully before completing the practical components of this Coursework.

2.1 Preliminary Steps

For this coursework you will should;

a) Create two EC2 Instances on AWS with the following parameters:
   Instance 1
   - Name = Build Server
   - Size = t3.large
   - OS = Ubuntu
   - Ports = 5000 open (optional)

   Instance 2
   - Name = Production Server
   - Size = t3.large
   - OS = Ubuntu
   - Ports = 5000 open
   - Storage = 12GB

b) Prepare for the practical Coursework task by:
   - Appropriately configuring Git and GitHub authentication on your Build Server (see lab 2 for guidance)
   - Installing Docker on the Build and Production Server (see lab 3 for guidance)
   - Install Ansible on the production server

3 Practical Tasks

The table below provides an outline of the steps required to complete the practical element of this coursework along with the marks available for each component. Marks indicated are for a complete working solution to the task and proportionate marks will be awarded for partial solutions. It is up to the group members to split the tasks per role (developer, quality assurance, operations, DevOps engineer) or work on all tasks together as a group.

Task	Description	Marks
1 (Developer)	**Building the Application** (a) Create a GitHub repository for the application called weather-ml-app and push the code files to it in the main branch. (b) Create an appropriate Dockerfile for the code you have been provided to create a Flask web application and push this to your repository. (c) Use this Dockerfile to manually create a Docker Image for this project and push this to DockerHub.	**(a) 2** **(b) 2** **(c) 2**
2 (Quality Assurance, developer)	**Testing the Application** Improve the app. At present, the app may have some errors and defects from initial development which causes the app to not function as intended. The QA member should work on the main repository branch to write three tests using unittest framework: (a) Smoke test (b) Unit test (c) Integration test Details on what the tests should achieve are included in the code files (smoke_test.py, unit_test.py, integration_test.py) in comments. **Tip! Run the test files one at a time and in order.** The developer should work on a new branch called "bugfix" to address any issues picked up by the tests above. (d) Fix the code in the "bugfix" branch. (e) Run the tests again showing success. Note: these tasks should be done locally. Then push the new code from (a) to (d) to the remote repository. (f) Merge the bugfix branch into the main branch and push to remote repository.	**(a) 3** **(b) 8** **(c) 5** **(d) 10** **(e) 3** **(f) 3**
3 (Operations)	**Preparing the Production Server** On the Production Server, write and execute Ansible playbook(s) to; (a) Install Kubectl	**(a) 4** **(b) 2** **(c) 2** **(d) 2** **(e) 2**

	(b) Install and Start Minikube (c) Deploy your image from DockerHub to Kubernetes (d) Create a Service for your application (e) Scale your deployment appropriately to allow for Rolling Updates to be implemented in the future **This section can be completed manually without the use of Ansible for up to 50% of the available marks.**	
4 (DevOps Engineer)	**Configuring CI/CD Pipeline** Install and appropriately configure GitHub Actions workflows called "CI.yml" and "CD.yml" to automatically: (a) Detect changes to your GitHub Repository. (b) Install Flask and dependencies in requirements.txt. (c) Run the test files defined in Task 2, each in a separate step. (d) Build a Docker Image from your Dockerfile. (e) Test that a Container can be launched from your Image (i.e. Build Test – run a command inside the container to ensure that it has launched successfully. You do not need to do any functional testing here). **Hint!** Make sure your GitHub actions environment has docker set-up, you can also use docker run action. (f) Push your Image to DockerHub. (g) Deploy passed builds to Kubernetes without disrupting service. **Steps (a) to (e) should be in CI.yml file and step (f) in CD.yml file. Further 5 points are awarded if (f) is deployed in yml files.**	**(a) 2** **(b) 2** **(c) 2** **(d) 2** **(e) 6** **(f) 6** **(g) (5)(5)**

3.1 Practical Evidence

There are a number of evidence pieces required for the tasks of this Coursework which should be provided **in your Report**. Failure to provide **ANY** of these may result in marks not being awarded.

3.1.1 Final code

Include your completed code including the app, test, Dockerfile and your workflow codes.

3.1.2 Docker Evidence

You should provide a screenshot of your DockerHub showing that the Image has been pushed (from one member's account).

3.1.3 GitHub Evidence

You should provide a screenshot of your GitHub repository showing your project files have been pushed (from one member's account), both in 'main' and 'bugfix' branches with merging evidence, you can run the command: git log --graph in the main branch of Developer's terminal to obtain this.

3.1.4 Ansible Evidence

You should provide a screenshot of your Terminal showing that all stages of each of your Ansible playbooks pass. If you do not provide this evidence you will not be awarded the full marks based on the playbook alone.

3.1.5 All Ansible Playbooks

Include all Ansible Playbooks you have created, including the yaml files for deployment in task 4 (f).

3.1.6 GitHub Actions job logs

You should include a copy of the **FULL** GitHub Actions logs for your final job build. Provide a copy/pasted text of the full logs and an overall screenshot of the steps' success, make sure you include **your username and repository name** in the screenshot otherwise points will be deducted. You can download the log file from GitHub, by clicking on the settings icon on the right side of your job. See the screenshots below for reference.

3.1.7 Git log

An extract from your Git Log that clearly shows that you have utilized Version Control to track your changes throughout the tasks.

3.2 Video Demonstration

You should create a short video demonstration of your Continuous Integration/Continuous Deployment Pipeline (**see Recording Your Video**).

Your video should be no longer than 10 minutes in duration and **should be narrated** such that it is clear to the viewer that you understand the steps being carried out. One or multiple group members can narrate the video.

The video should be recorded **after** all of the practical tasks in **Practical Tasks** have been completed.

Before you start the video, you should ensure that your set up and pipeline are ready.

During your video you should:

- Use the 'curl $(minikube ip):NODE_PORT' command on the Production Server to show the deployed application in the container is running correctly. Use a separate terminal to the one that is running the GitHub actions runner.

- Go to the browser and show the application running using the Production server's public IP address followed by the container port number.

- Update the code line 9, under weather_classes variable in app.py, from '**`rainy`**' to '**`rain`**' and image version to v2, then push this change to GitHub.

- Review the full GitHub Actions job output of this build. When viewing the output, you should clearly indicate where there is evidence of every step in your pipeline.

- Go back to the second terminal connected to your production server and use the 'curl $(minikube ip): NODE_PORT' command again to show that the new application version has been successfully deployed.

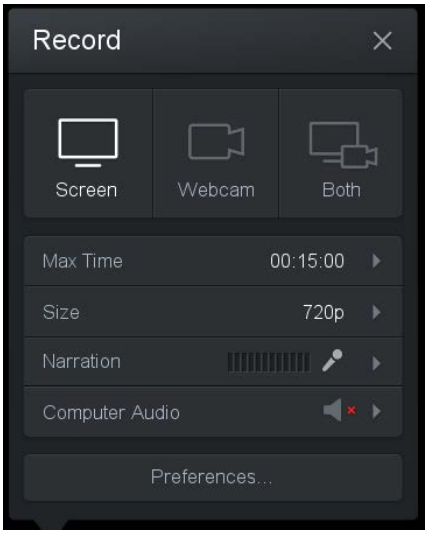
- Go to the browser and refresh it to show the application is still running after the update.

3.3 Recording Your Video

You can use the following free tool to record your video:
https://screencast-o-matic.com/screen-recorder

Click the 'Launch Free Recorder' button and follow the instructions to get started.

When the application launches you should see a control panel like this:



Select 'Screen' from the top three icons and ensure that your Microphone us selected under the 'Narration' option.

You can adjust the size of the recording area by dragging the black and white border to adjust the window size or you can set a specific size using the 'Size' option.

Save your video as an MP4 file (**other formats will NOT be accepted**) with the following naming convention: GroupNumber__DevOps_CW.mp4

This video should be either uploaded as a file to the video submission portal (preferably) or uploaded to your GCU provided OneDrive account and a shared link (**accessible by everyone with no expiry**) to the video.

Please note that links to other cloud services WILL NOT BE ACCEPTED.

If your video is not accessible to everyone or expires it may not be available for marking/moderation and could result in loss of marks.

4 Appendix 1 – Report Introduction marking rubric [20]

Element	Poor (0-39%)	Average (40-49%)	Good (50-59%)	Very Good (60-69%)	Excellent (70-100%)
DevOps pipeline	Poor or no explanation on the rationale for the CI/CD pipeline.	Limited explanation on the rationale for the CI/CD pipeline.	Good explanation on the rationale for the CI/CD pipeline but lacks detail.	Very good explanation on the rationale for the CI/CD pipeline covering most steps.	Excellent explanation on the rationale for the CI/CD pipeline covering all aspects
DevOps Principles	Poor or no link to DevOps principles.	Limited link to DevOps principles.	Good link to DevOps principles.	Very good link to DevOps principles with details.	Excellent link to DevOps principles thoroughly covering all principles.
Pipeline challenges	Poor or no implementation challenges mentioned.	Limited implementation challenges mentioned.	Good implementation challenges discussed.	Very good implementation challenges discussed with reasoning.	Critical analysis of implementation challenges presented.
Challenge reflection/action	Poor or no reflection on the challenges.	Limited reflection on the challenges with no actions identified to overcomes these.	Good reflection on the challenges with some actions identified to overcomes these.	Very Good reflection on the challenges with proper actions identified to overcomes these.	Excellent reflection on the challenges with all actions identified to overcome them.
Testing and code explanation	Poor or no code explanation, test failure, reason/bug source.	Limited code explanation, test failure, reason/bug source, with no potential fix proposed.	Good code explanation, test failure, reason/bug source, and attempt at some fixes.	Very Good code explanation, test failure, reason/bug source, and most issues were fixed.	Excellent code explanation, test failure, reason/bug source, and all issues were fixed.